IoT-Enabled Plant Health Monitoring tool using Machine Learning

Documentation Report by Madan

March 2, 2026

0.1 Aim of the Project

The objective of this project is to develop an IoT-enabled embedded system for the continuous monitoring of plant health. This is achieved by incorporating the cloud deployment of the ML Model. The model taken an image of the plant leaf and then tell the status of the health of the plant in the three categories: **Healthy**, **Moderate** or **Infected**.

0.2 Components and Technologies Used

0.2.1 ESP32 Microcontroller

The ESP32 is a high-performance, IoT-enabled microcontroller capable of wireless data transmission to cloud servers. It serves as the primary controller for:

- Interfacing with sensors via General Purpose Input/Output (GPIO) pins.
- Utilizing an Integrated Analog-to-Digital Converter (ADC) to transform continuous voltage measurements from sensors into discrete digital values.

0.2.2 Sensors and Hardware

The system integrates various sensors to capture the physical state of the plant environment.

Capacitive Soil Moisture Sensor

The capacitive sensor measures soil moisture by detecting changes in voltage caused by dielectric variations. Water molecules increase the soil's ability to store charge; these fluctuations are processed through the ESP32 ADC to determine moisture levels.

DHT11 Temperature and Humidity Sensor

The DHT11 provides digital environmental readings with the following specifications:

Parameter	**Range / Accuracy**
Communication Type	8-bit Single-bus Digital Output
Humidity Range	20–90% RH (±5% Accuracy)
Temperature Range	0–50°C (±2°C Accuracy)

Table 1: DHT11 Sensor Specifications

0.3 Machine Learning for Classification

Rather than developing a model from the ground up, this project utilizes **Transfer Learning** by fine-tuning a pre-trained classification model.

0.3.1 Model Generation

- **Teachable Machine:** Used to train the initial classification logic on a self-curated dataset of plant leaves.
- **TensorFlow/Keras:** The framework used for implementing and exporting the model into the `.h5(hierarchical Data Format)` format.

0.3.2 Deployment and Model Loading

The model is loaded into the cloud environment using the following implementation:

```
import tensorflow as tf
# Loading the fine-tuned model
model = tf.keras.models.load_model("keras_model.h5")
```

0.4 System Architecture

0.4.1 Hub and Spoke System

The architecture follows a **Hub and Spoke** model:

- **Spoke:** The ESP32 handles lightweight edge tasks, such as physical sensor data collection.
- **Hub:** The Hugging Face cloud server provides the computation ability to the model. without making model to be dependent on the user's Machine.

0.4.2 Cloud Deployment (Hugging Face & Flask)

1. **Constraint of the ESP:** Since, Esp-32 cannot handle the model computation due to memory and processing power constraints.
2. **Spoke & Hub System:** The esp-32 (**spoke**) handles the lightweight physical sensors and the powerful cloud server (**hub**) for AI math. (*Healthy, Moderate, or Infected*).

0.5 Methodology

0.5.1 Setting up the Physical Connection in the Embedded Systems

The initial phase involves establishing the hardware architecture using an ESP32 microcontroller. The connections include:

1. **16-pin LCD Display:** Connected to keep watch on details read through the sensors. A 4-pin data transfer method (d_4, d_5, d_6, d_7) was utilized for transferring information from the ESP32 to the LCD.
2. **Soil Moisture Sensor:** Integrated to monitor the hydration levels of the plant environment.

3. **DHT11 Sensor:** Connected to monitor ambient temperature and humidity.

0.5.2 Machine Learning Model Development

Google's Teachable Machine was utilized to create the required classification model. The underlying framework used is TensorFlow, which refrains us from getting into extensive code writing by providing a massive collection of pre-written codes.

The steps involved in acquiring the model files are as follows:

1. **Fine-Tuning:** Fine-tuning the pre-trained model through self-collected data of plant leaves across different health categories.

2. **Exporting the Model:** Exporting the model through Keras. A Keras file is received, which can be loaded into the environment using just a few lines of code.

0.5.3 Deployment to Cloud Using Hugging Face

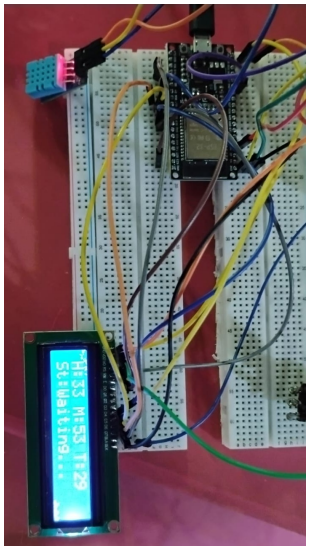
The deployment architecture facilitates communication between the hardware and the cloud environment.

- **Python Web App (Flask):** We built a web server that waits for traffic. It acts as a middleman between the phone/dashboard and the hardware.

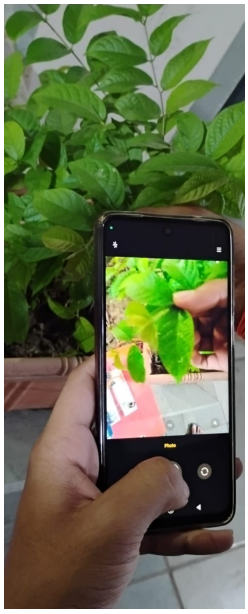
- **Sending Data UP (GET Request):** Every 5 seconds, the ESP32 takes its sensor numbers and attaches them to a URL, visiting our Hugging Face website. The Python server grabs those numbers and updates the web dashboard.

- **Getting Data DOWN:** At the same time, the Python server replies to the ESP32 with a single word representing the current AI prediction (e.g., "Infected").

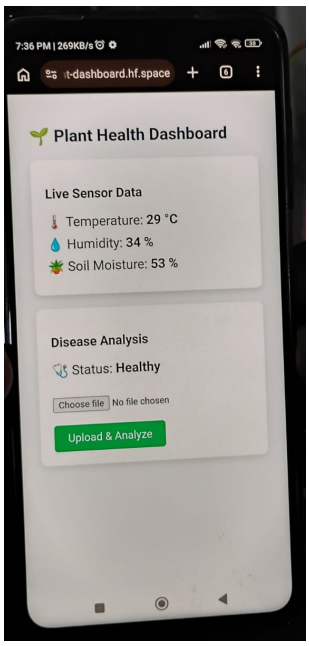
0.6 Observations



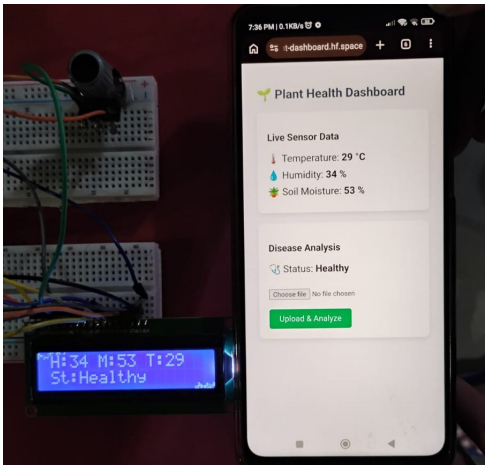
(a) The embedded system



(b) Capturing Sample to test



(c) Dashboard



(d) View of the realtime result on the web

Figure 1: Experimental observations of the Plant Health Dashboard system.

0.7 Results

The project successfully demonstrated:

- IoT communication between hardware and self deployed cloud platform.
- Real-time classification using fine-tuned ML models.
- **Future Scaling and Modification:** A fully functional end-to-end monitoring pipeline could be developed by elevating the dataset and also using the sensor data to be an active feature in the classification. This will result in the very useful insights in the Agriculture related fields.
- `https://prince12313-plant-dashboard.hf.space/` The hosted dashboard.

0.8 The Bibliography

- Teachable Machine, Google.
- TensorFlow Open Source Framework.
- hf Hugging Face Cloud Platform.
- Flask Web Framework for Python.